

Hardware-in-the-Loop Simulation of a UAV Altitude Control System via FM Telemetry Link

Abdul Muqtadir
Dept. of Avionics Engineering
NUST College of Aeronautical Engineering
Risalpur, Pakistan

Rehan (23101012 - AC)
Dept. of Avionics Engineering
NUST College of Aeronautical Engineering
Risalpur, Pakistan

Haroon (23101009 - AC)
Dept. of Avionics Engineering
NUST College of Aeronautical Engineering
Risalpur, Pakistan

Anas (23101010 - AC)
Dept. of Avionics Engineering
NUST College of Aeronautical Engineering
Risalpur, Pakistan

Abstract—This project focuses on designing and testing an unmanned aerial vehicle (UAV) altitude control system. We built a custom Python block to simulate second-order plant dynamics controlled by a Proportional-Integral-Derivative (PID) loop. The flight telemetry was then frequency modulated (FM) and transmitted over a real 915 MHz radio frequency (RF) link using an ADALM-Pluto Software-Defined Radio (SDR). At the receiver end, we used Quadrature Demodulation and a Single Pole Infinite Impulse Response (IIR) filter to reject Additive White Gaussian Noise (AWGN) and recover the autopilot feedback signal. The final results confirm that our closed-loop telemetry setup works over the air and can handle high-interference environments.

Index Terms—UAV, Telemetry, Software-Defined Radio, PID Control, Hardware-in-the-Loop, DSP, GNU Radio

I. INTRODUCTION

Before putting new avionics systems into actual flight, they must undergo strict testing. Physical flight tests are expensive and carry significant safety risks. Hardware-in-the-Loop (HITL) simulations help solve this problem by connecting software models directly to physical hardware prototypes. For this project, our goal was to build a complete HITL telemetry pipeline. We generated real-time flight dynamics in software, sent that data over a physical RF link, and applied digital signal processing (DSP) to recover the signal despite atmospheric interference.

II. LITERATURE REVIEW

UAV altitude stabilization is theoretically based on classical control theory. Nevertheless, the industry standard in avionics continues to be the Proportional-Integral-Derivative (PID) controller due to its reliability, computational efficiency and robust disturbance rejection characteristics [1], despite the advances in modern predictive control.

Integration of Software-Defined Radios (SDR) has revolutionized aerospace testing methodologies in the telemetry validation domain. SDR platforms like the ADALM-Pluto provide engineers with a means of rapid prototyping of complex RF data links and testing simulated telemetry against real-world atmospheric channel impairments without the need for

dedicated analog hardware [2]. In addition, the use of open source DSP frameworks like GNU Radio allows to directly integrate the physical plant dynamics in the modulation chain mathematically. This results in a highly accurate digital twin of the vehicle's mechanics and communication link, simplifying the transition from simulation to physical deployment [3].

III. SYSTEM ARCHITECTURE

Our setup forms a complete closed-loop transmission cycle. The main parts of the system are a custom physics engine, an FM transmitter chain, the actual RF environment, and a receiver equipped with DSP blocks. The full system flowgraph is shown in Fig. 1.

A. UAV Plant Dynamics & PID Control Logic

We modeled the UAV mathematically as a second-order system that accounts for mass and aerodynamic drag. The control signal (thrust), referred to as $u(t)$, is generated by a PID controller:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (1)$$

We tuned the gains specifically to $K_p = 3.0$, $K_i = 0.08$, and $K_d = 0.9$. This forces an underdamped step response, which let us visually verify system stability by looking at the overshoot and settling times on our scopes.

The core logic for the controls and physics is provided in the Python block below:

```
1 # PID Control Loop
2 error = self.target_alt - self.altitude
3 self.integral += error * self.dt
4 self.integral = max(min(self.integral, 300.0), -300.0)
5
6 derivative = (error - self.prev_error) / self.dt
7 self.prev_error = error
8
9 # Control Signal (Thrust)
10 u = (self.Kp * error) + (self.Ki * self.integral) + (self.
    Kd * derivative)
11
12 # Plant Dynamics (Second-order system)
13 mass = 1.0
14 drag = 1.1
```

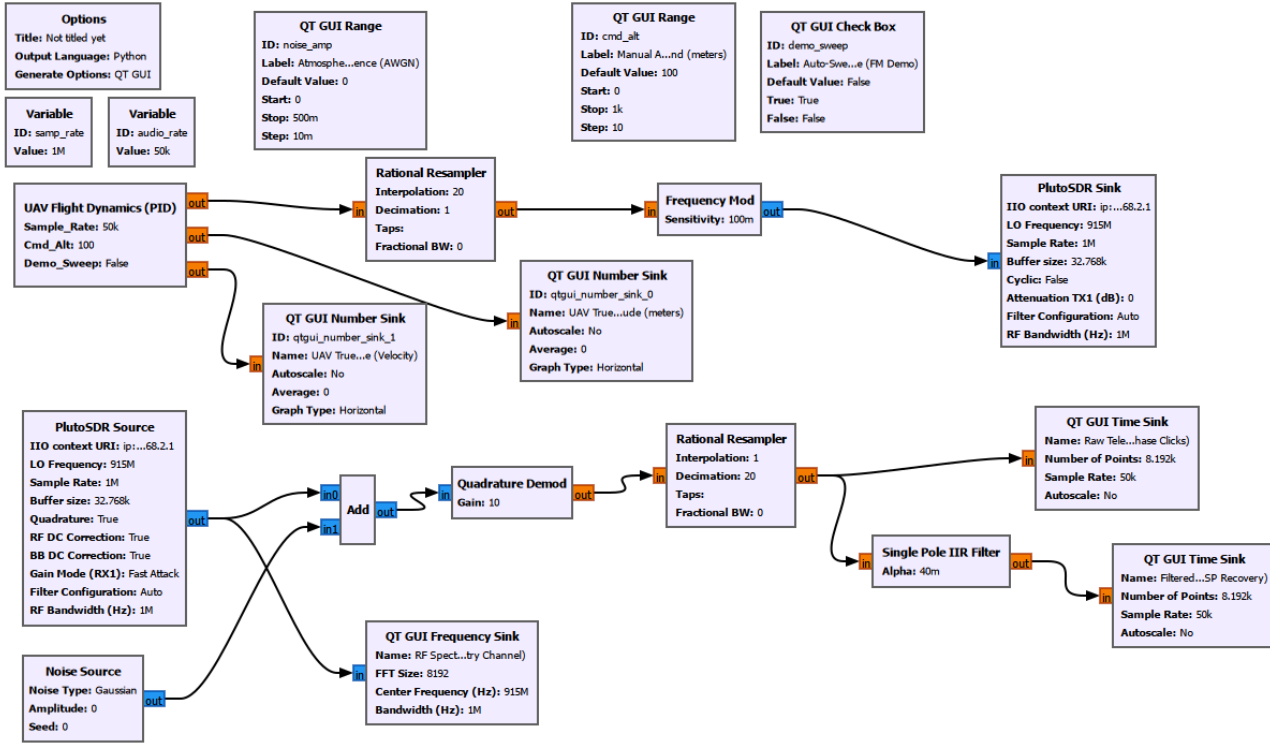


Fig. 1. Complete GNU Radio Block Diagram of the UAV Telemetry System, illustrating the transmission, RF hardware loopback, and DSP recovery chains.

```

15 acceleration = (u - (drag * self.velocity)) / mass
16
17 # Physics Integration
18 self.velocity += acceleration * self.dt
19 self.altitude += self.velocity * self.dt
20 self.altitude = max(0.0, self.altitude) # Ground clamp

```

Listing 1. Python Implementation of the PID Controller and Second-Order Plant Dynamics

IV. WORKING PROCEDURE

We broke down the development and testing of the telemetry system into four main steps:

1) *Phase 1 - Software Simulation Setup:* We wrote an Embedded Python Block in GNU Radio to act as the UAV flight computer. We applied a time-warp factor of 80 to the physics engine so the 800-meter step maneuver could be clearly seen happening in real-time. The block has three outputs: a normalized FM transmission stream, true altitude in meters, and true vertical velocity in m/s.

2) *Phase 2 - FM Modulation and RF Loopback:* The normalized telemetry data goes into a frequency modulator. We interpolated and upconverted this baseband signal, then sent it to the ADALM-Pluto SDR Sink. The SDR transmitted the data over the air at 915 MHz. Our RX antenna picked it right back up, creating the physical hardware loop (Fig. 2).

3) *Phase 3 - DSP and Noise Injection:* After receiving the physical RF signal, we routed it into our digital receiver chain. To test how the system handles bad conditions, we added an Additive White Gaussian Noise (AWGN) source to the channel. Next, we demodulated the signal using a

Quadrature Demodulator. Finally, we added a Single Pole IIR filter ($\alpha = 0.04$) to filter out high-frequency phase clicks and bring back the underlying low-frequency control curve.

4) *Phase 4 - GUI Dashboard Configuration:* We built a graphical user interface (GUI) using QT GUI blocks to serve as our ground-control station. This interface features an RF spectrum analyzer, side-by-side time scopes for comparing raw and filtered telemetry, and manual flight controls.



Fig. 2. ADALM-Pluto SDR utilized for physical RF loopback at 915 MHz.

V. RESULTS AND ANALYSIS

To see how well the system held up, we commanded the flight computer to perform a step response while we varied the channel conditions.

A. Baseline Flight Response

Under perfect conditions (AWGN set to zero), the UAV held a steady altitude. Looking at the FM spectrum, the signal stayed tightly grouped around the center frequency during the hover, which confirmed our baseband signal was clean (Fig. 3).

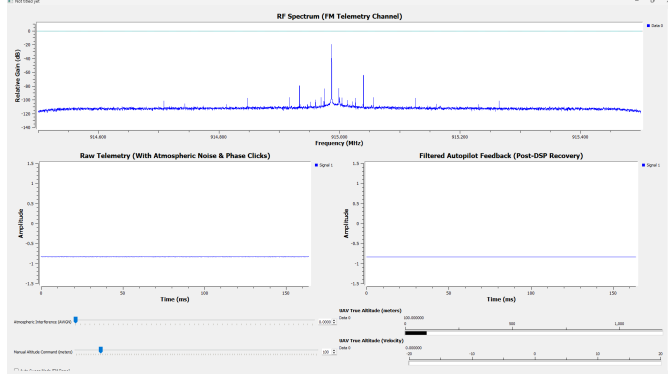


Fig. 3. Baseline steady-state hover with no atmospheric noise.

B. Dynamic Response and Noise Rejection

We then triggered an automated step command that forced the UAV to climb from 200m to 1000m. At the same time, we cranked up the AWGN in the receiver path to simulate heavy atmospheric interference.

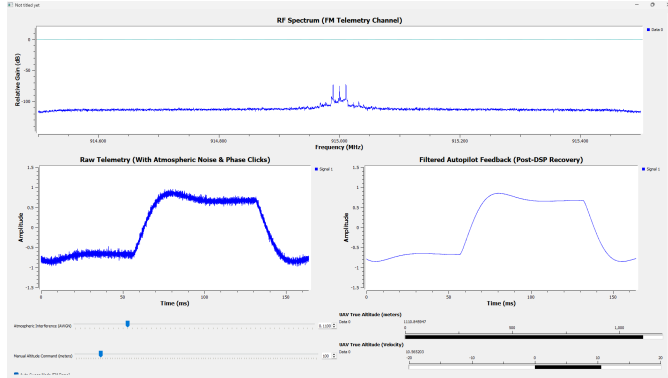


Fig. 4. Underdamped step response during Auto-Sweep mode. Note the FM sideband expansion on the spectrum analyzer and the noise rejection on the right time scope.

As shown in Fig. 4, the raw telemetry on the left was completely distorted by the noise floor. However, our DSP filter rejected the high-frequency noise perfectly, giving us a smooth, readable underdamped curve on the right. In addition, the spectrum analyzer clearly showed the instantaneous frequency deviation expanding as the UAV rapidly climbed.

VI. CONCLUSION

By combining a custom Python script with the ADALM-Pluto SDR, we successfully showed how avionics telemetry and HITL simulations work in practice. The PID controller provided a very stable physical response, and using quadrature

demodulation alongside IIR filtering proved to be a highly effective way to recover flight data from a noisy RF channel.

REFERENCES

- [1] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 7th ed. Boston, MA, USA: Pearson, 2014.
- [2] D. Pu and A. M. Wyglinski, *Digital Communication Systems Engineering with Software-Defined Radio*. Norwood, MA, USA: Artech House, 2015.
- [3] E. Blossom, "GNU Radio: Tools for Exploring the Radio Frequency Spectrum," *Linux Journal*, vol. 2004, no. 122, p. 4, Jun. 2004.